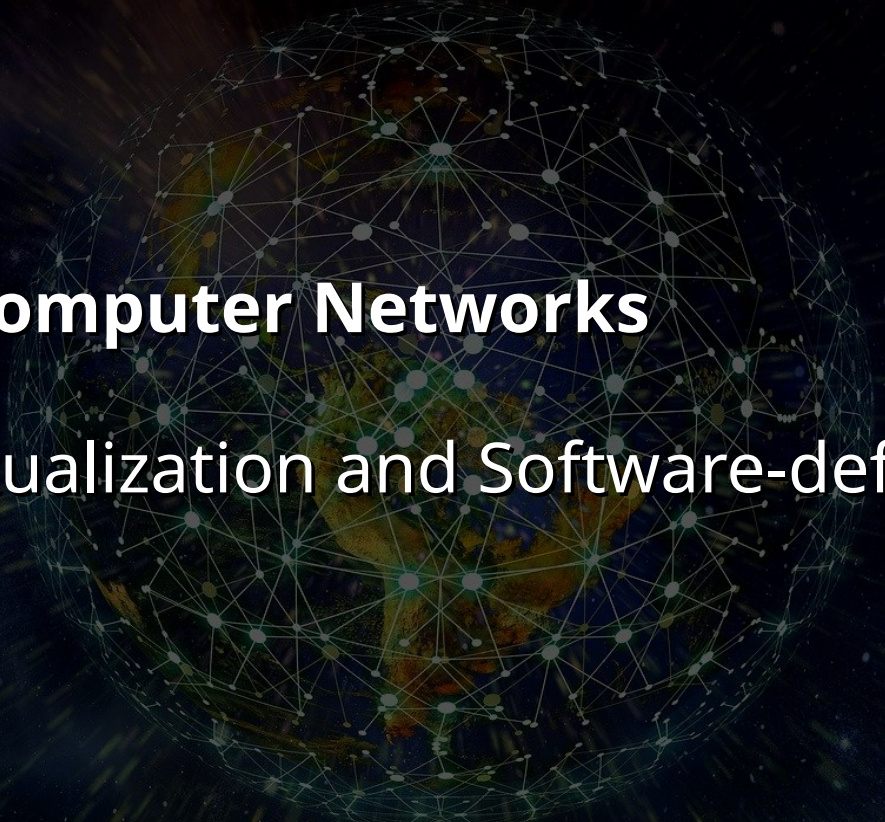


The background of the slide features a central image of the Earth, where the continents are visible in brown and green against a blue ocean. The Earth is overlaid with a complex network of white lines and dots, representing a global computer network. The entire scene is set against a dark, starry space background with streaks of light radiating from the center.

Advanced Computer Networks

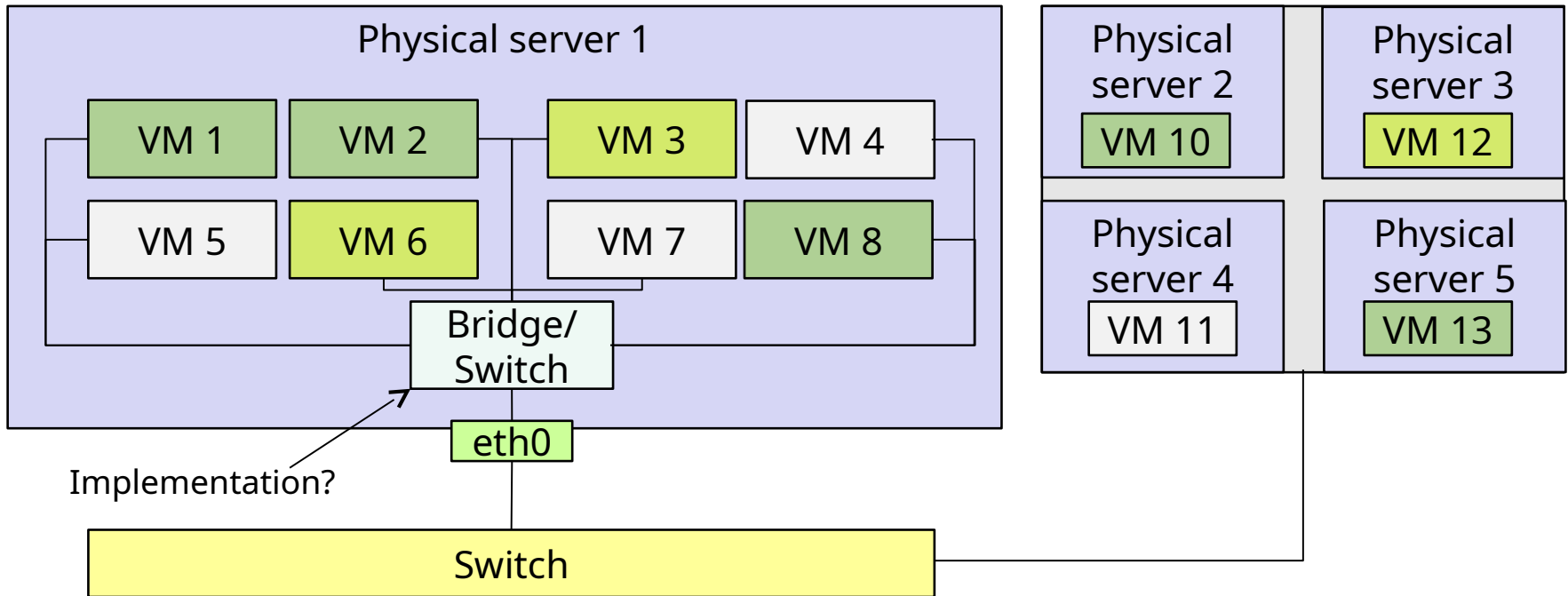
Network Virtualization and Software-defined Networking

Winter term 2025/26

marius.feldmann@tu-dresden.de

Issues with virtual infrastructures

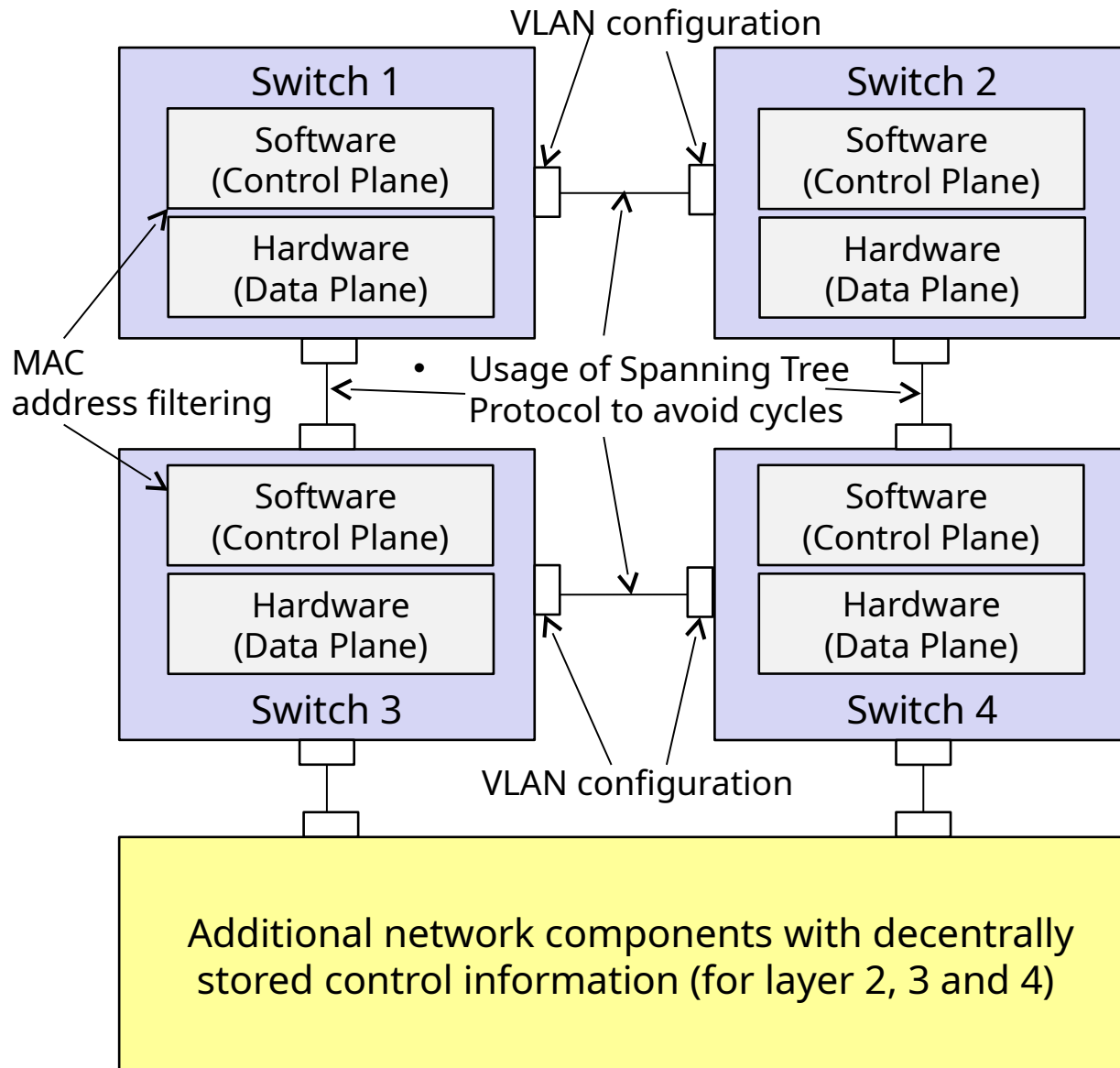
VM n ...VM of tenant n
VM m ...VM of tenant m



- Switch caches MAC addresses of all virtual machines → scalability issues
- Regular VLAN setups used for separating several networks between VMs from each other (e.g. networks from different tenants) are too little flexible (and they do not scale)

→ **Overlay networks** are created on top of network infrastructure

Issues with regular layer 2 networks



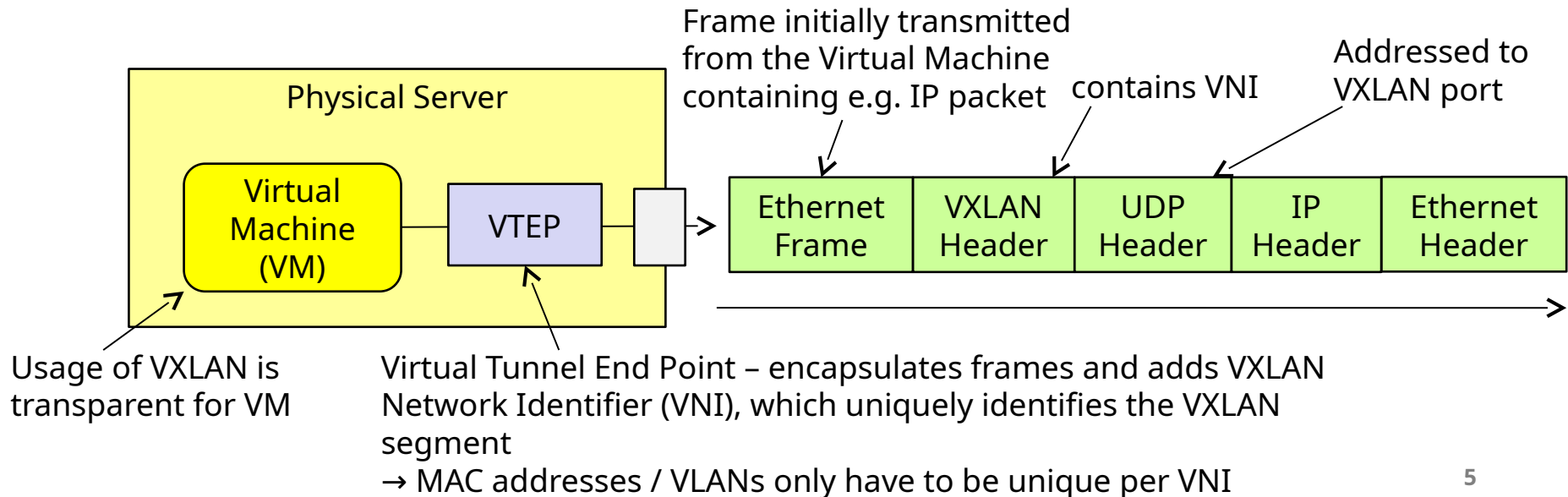
- Administrators have to configure several protocols
- Configuration is extensive and error-prone
- Distributed configuration has to be consistent
- Switches do not have global knowledge about network → global optimization is not possible
- No cross-layer optimization

Outline

1. VXLAN
2. EVPN
3. VRF
 - Motivation
 - VRF-Lite
 - VRF example
4. Software-defined Networking
 - Overview
 - Northbound / Southbound API
5. OpenFlow
 - Overview
 - Flow tables
 - Packet processing
6. Open vSwitch
7. Open Virtual Network

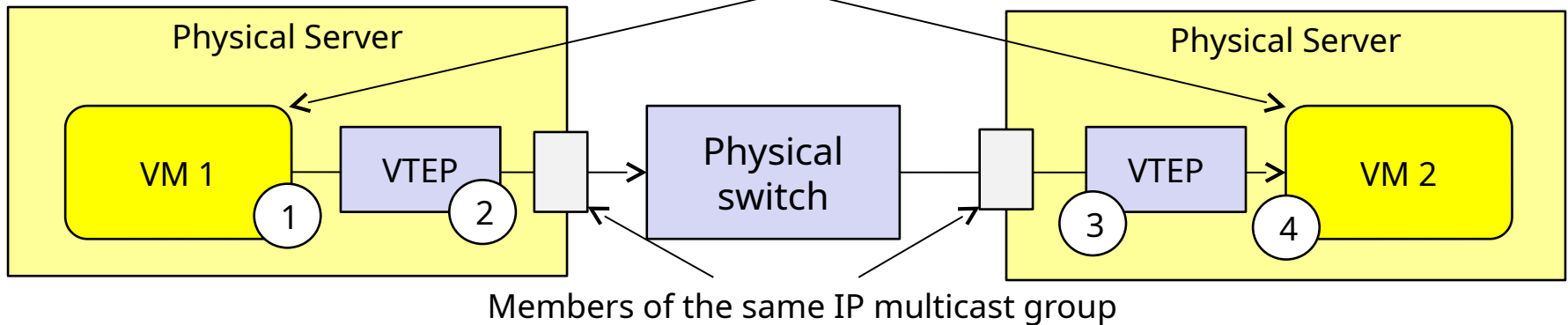
VXLAN - Overview

- Goal of Virtual Extensible LAN (VXLAN) is to introduce overlay networks to avoid scalability issues in huge virtualized infrastructures
- Specification provided as "informational RFC" (RFC 7348)
- Basic approach:
 - Layer 2 information is transmitted via a Layer 3 network only to those nodes executing e.g. VMs within a relevant layer 2 overlay network (= VXLAN segment)
 - Ethernet frames on the virtual network layer are encapsulated into VXLAN and UDP packets – VXLAN header information contains identifier for VXLAN segment (=VNI)



VXLAN

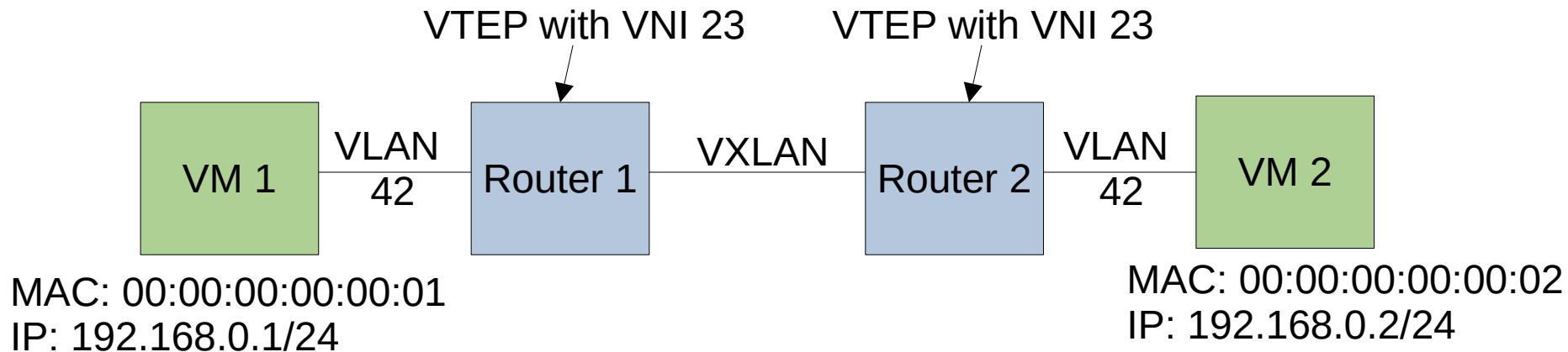
VMs are located in the same virtual L3 subnet



- ① • VM 1 sends ARP request as the destination (VM 2) is located in the same IP subnet
- ② • VTEP adds VXLAN VNI and sends the frame to the IP multicast address associated to the VNI, the packet is transmitted via UDP to the VXLAN port (default 4789)
- ③ • VTEP on the receiving server, executing the addressed VM receives UDP datagram and removes added headers → only initial Ethernet frame transmitted by VM 1 remains
• VTEP caches mapping of source MAC address of the encapsulated MAC address and the source IP of the encapsulating IP packet → reply can be done as IP unicast
- ④ • Ethernet frame is handed over transparently to the receiving VM which is part of the relevant VXLAN segment

EVPN

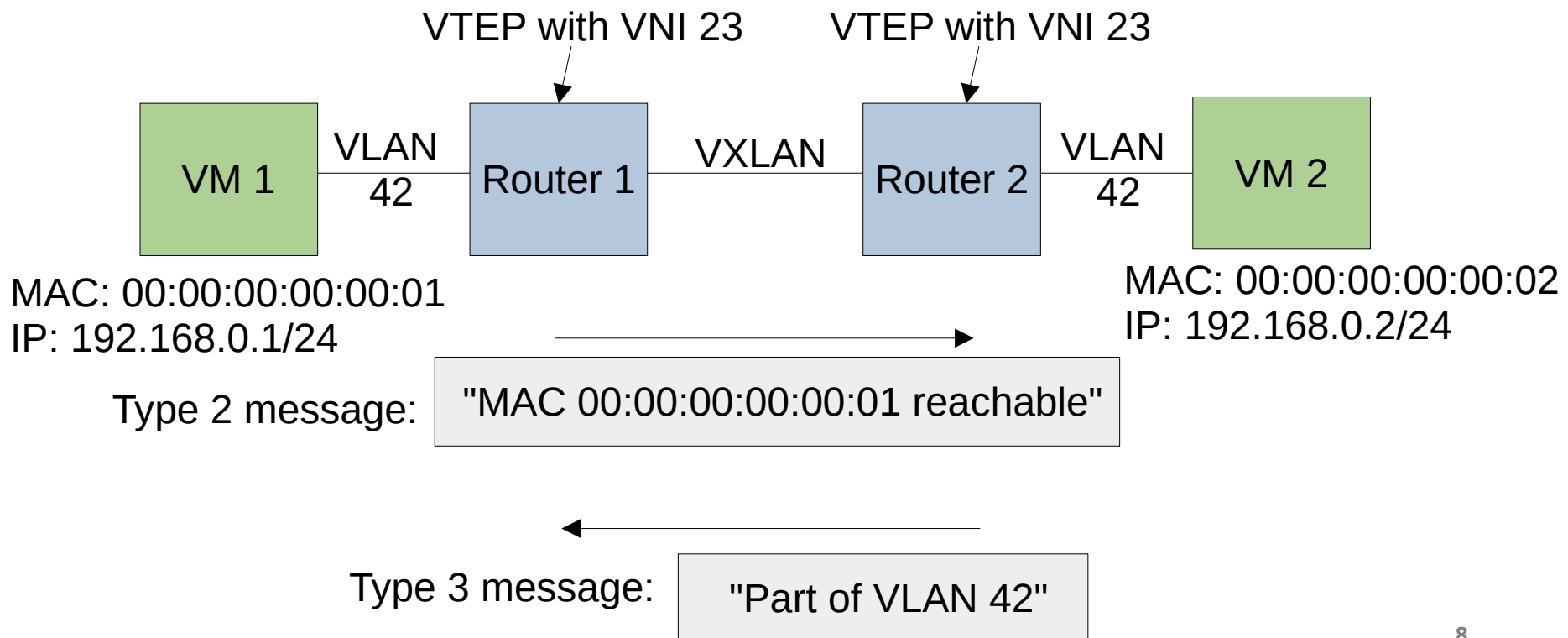
- VXLAN (as other approaches such as NVGRE or GENEVE) offers a possible data path for overlay networks without providing a solution for the control plane
- EVPN is an extension of BGP to distribute MAC reachability information e.g. for VXLAN setups



- Without any management solution, the following issues exist among others in the sketched scenario:
 - Router 1 is not aware of L2 accessibility of MAC address 00:00:00:00:00:02 via VXLAN with VNI 23
 - Router 1 has no information that Router 2 has interfaces to a shared broadcast domain
- Manual configuration is required ☹️

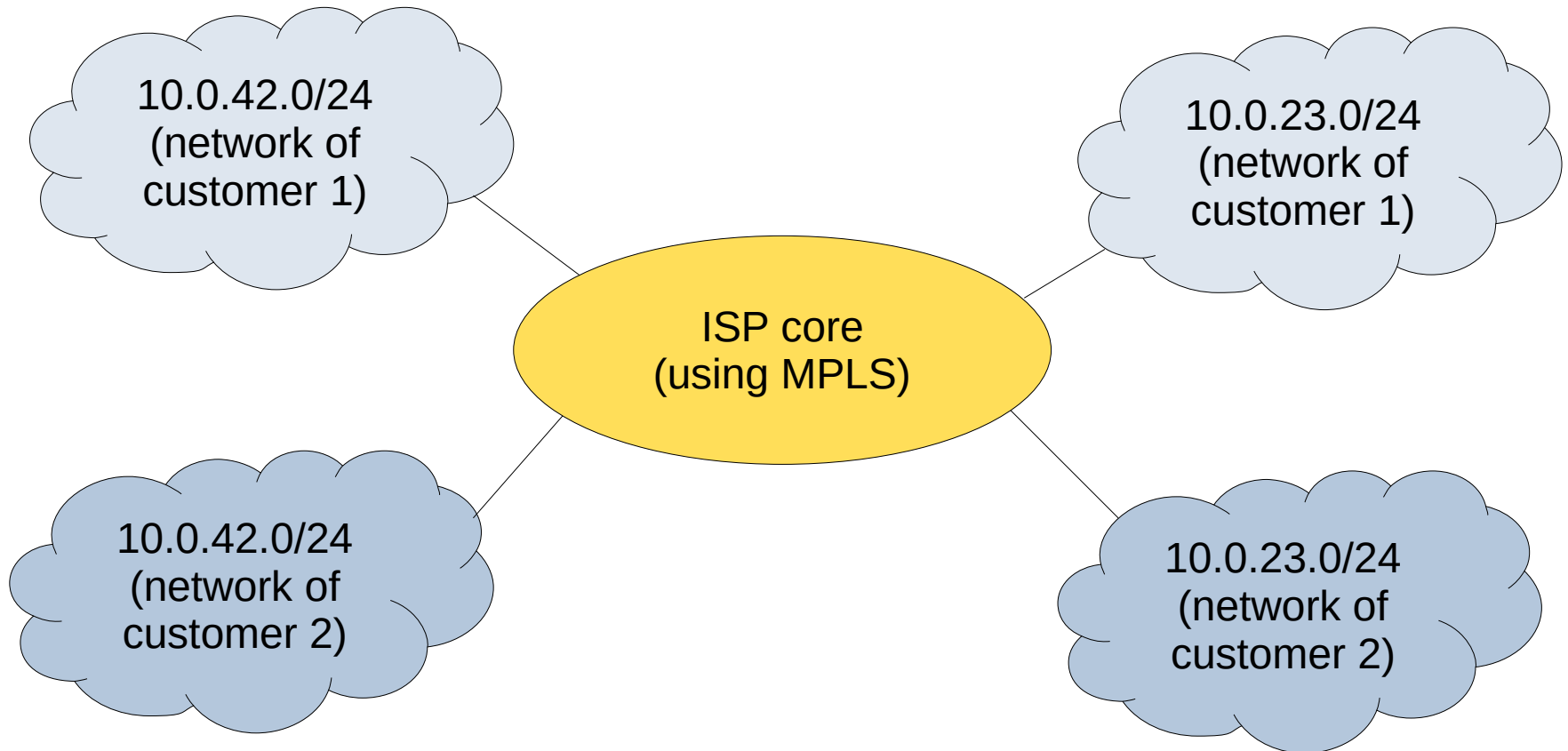
EVPN

- EVPN introduces BGP Network Layer Reachability Information (NLRI) of different types ("Route Types") for managing the control plane of an overlay network
 - **Type 2: MAC/IP Advertisement route**
 - **Type 3: Inclusive Multicast Ethernet Tag route**



VRF motivation

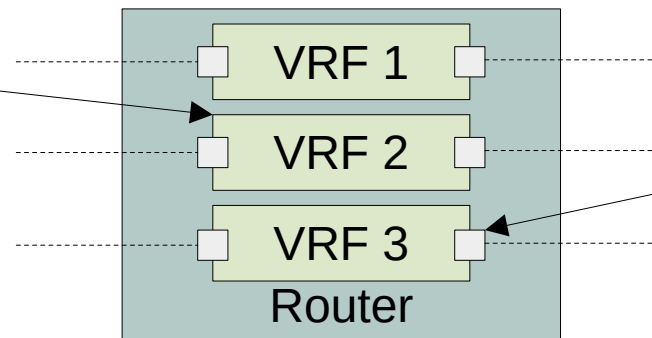
- In Internet Service Provider or data center networks it is often required to interconnect IP networks via a backbone / core infrastructure used for several customers / tenants
→ conflicts of overlapping IP address ranges may appear



VRF overview

- Virtual Routing and Forwarding (VRF) makes it possible to define several logically separated virtual routers on a single system (→ L3 traffic separation)
- Two core mechanisms:
 - 1) Interface partitioning and virtualizing of routing/forwarding,
 - 2) Path isolation and virtualizing of interconnects

For each VRF, a dedicated Forwarding Information Base exists (as well as potentially a dedicated Routing Information Base)



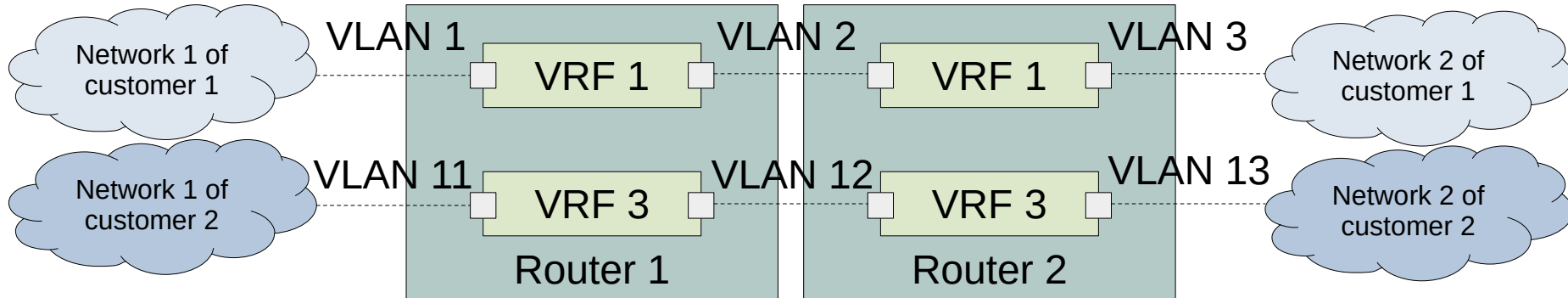
Each VRF has a set of interfaces via which packets are received and transmitted to connected networks.

- Sockets can be bound to the VRF device in order to make processes (e.g. routing daemons) work within the VRF
- Simple example for VRF setup under Linux:

```
# ip link add vrf1 type vrf table 42
# ip link set dev vrf1 up
# ip link set dev eth1 master vrf1
...
# sudo ip route add 192.168.100.0/24 via 192.168.2.1 table 42
```

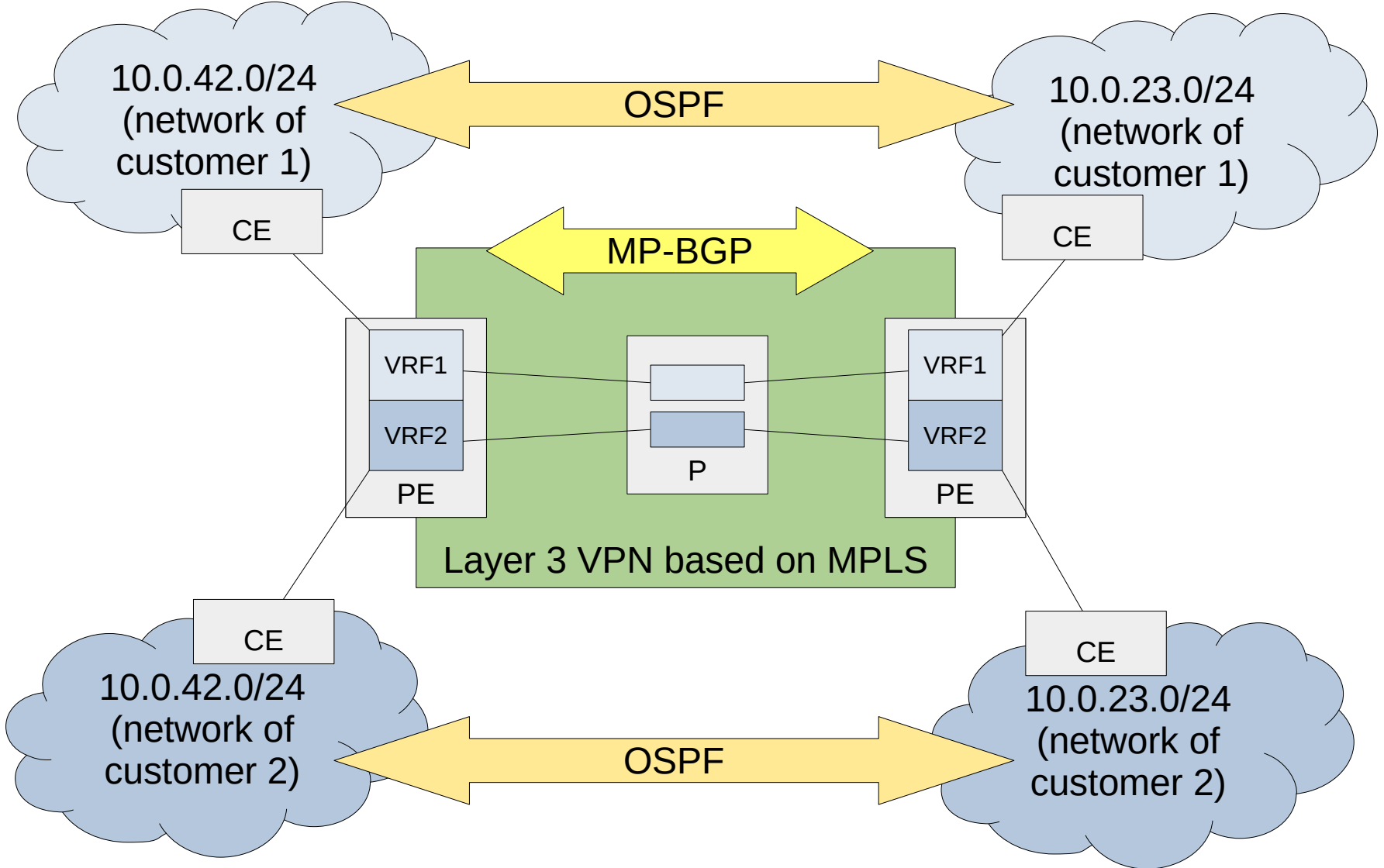
VRF-Lite

- Path isolation can be achieved using various approaches, e.g.:
 - VLANs, Generic Routing Encapsulation (GRE) tunnels, VXLANs, MPLS VPNs
- If path isolation is not done via an IP VPN, the term "VRF-Lite" may be used to point out that interface partitioning and virtualization of routing/forwarding is done but that path isolation is set up in a simple manner with limited flexibility



- Apart from flexibility, scalability of VRF-Lite is limited (specifically in the case VLANs are used for path separation as VLAN IDs are just 12 Bit identifiers)

VRF example

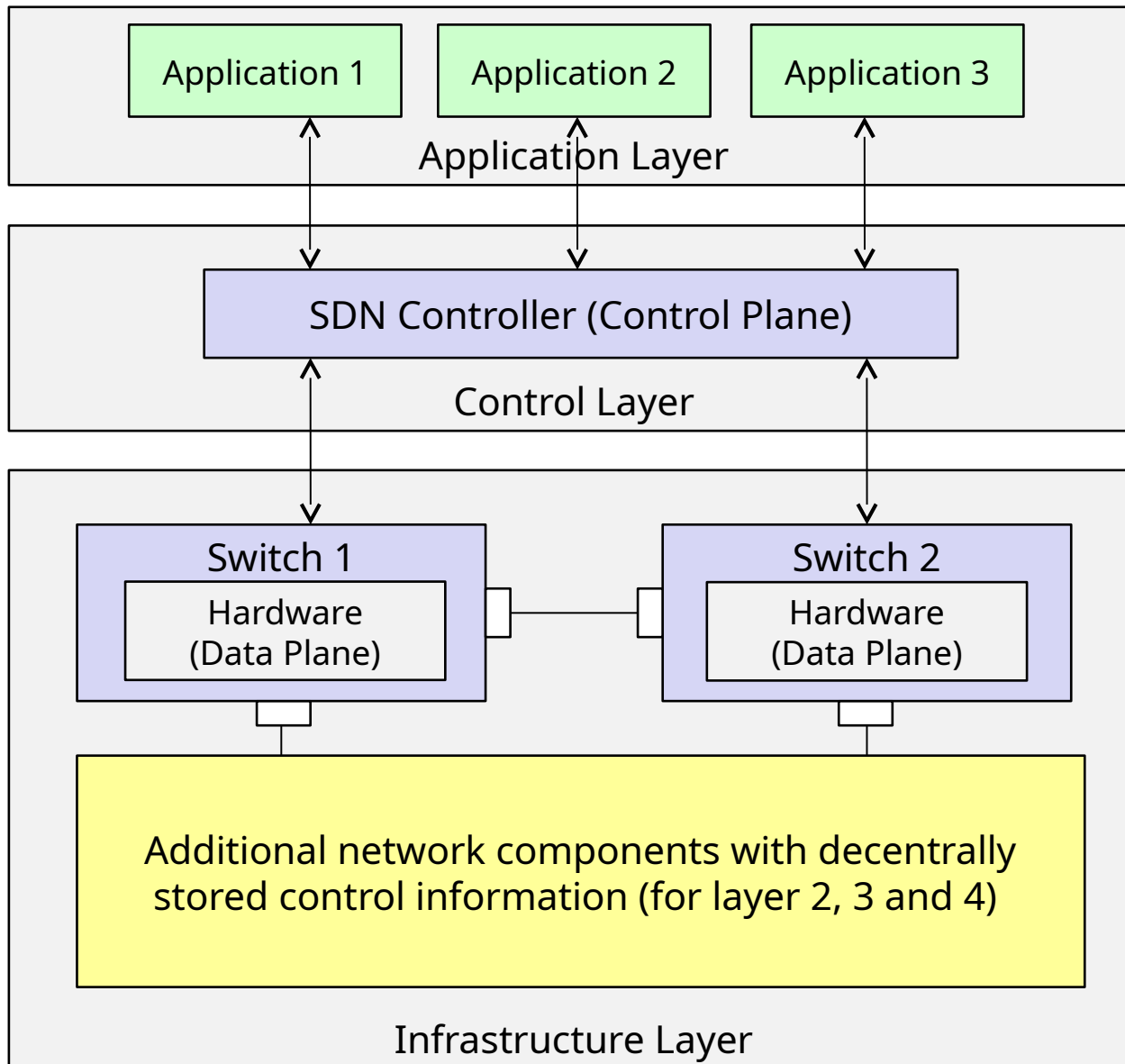


CE...Customer Edge Router
MP-BGP...Multiprotocol BGP

PE...Provider Edge Router

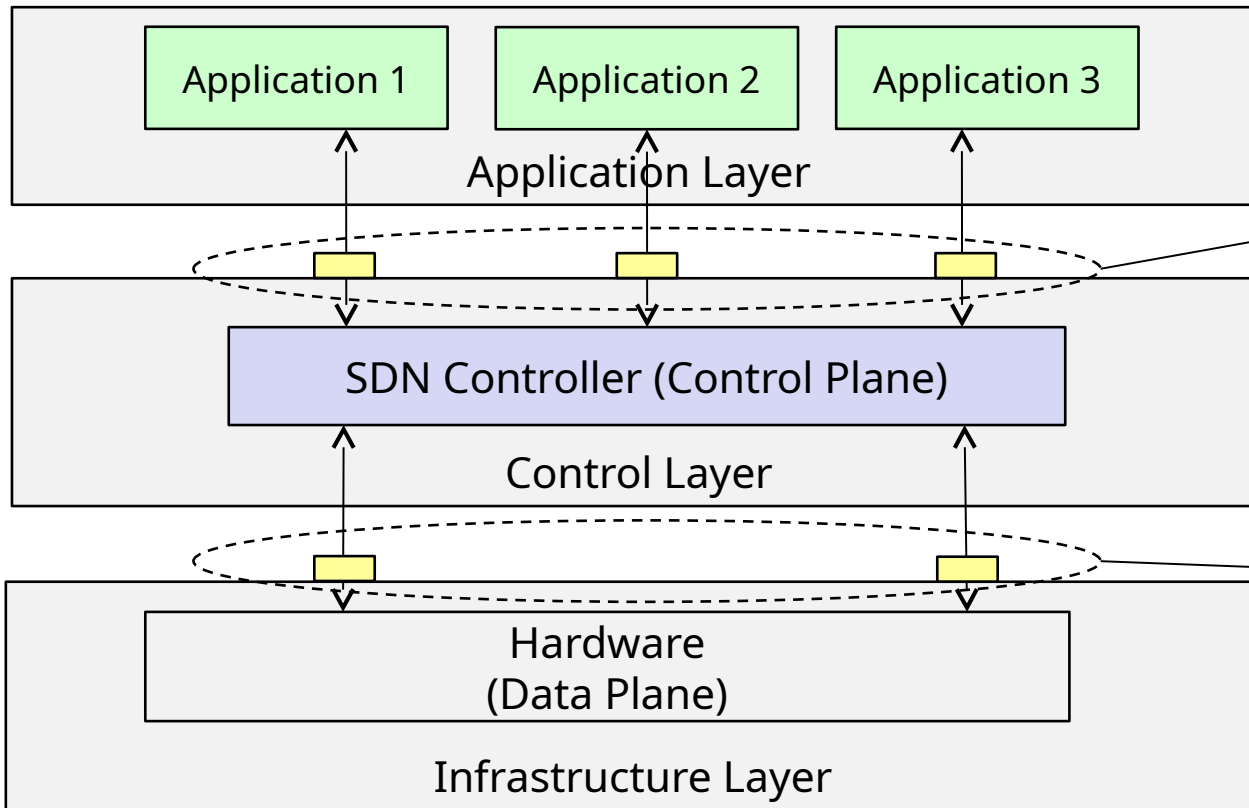
P...Provider Router

Software-defined Networking - Overview



- SDN is a networking approach characterised by handing over the control of the data flows and of the packet forwarding to central nodes in the network ("SDN controller") and the separation of this control plane from the data plane
- Approach is based on work by UC Berkeley and Stanford University

Northbound / Southbound API



Northbound API

- API for applications making use of the control plane functionality
- E. g. Used by:
 - Firewalls
 - Load-Balancers

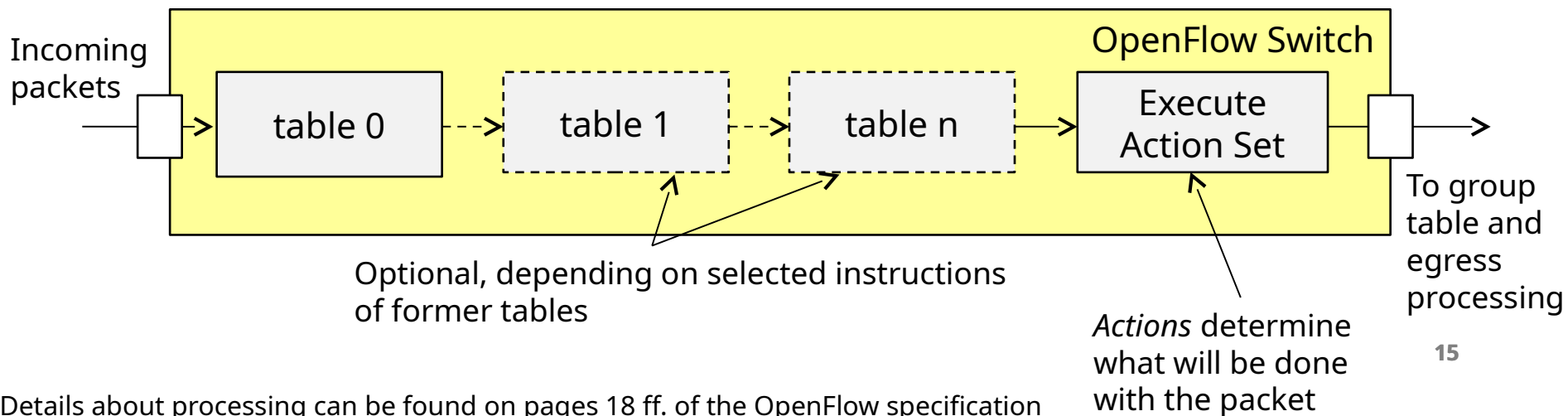
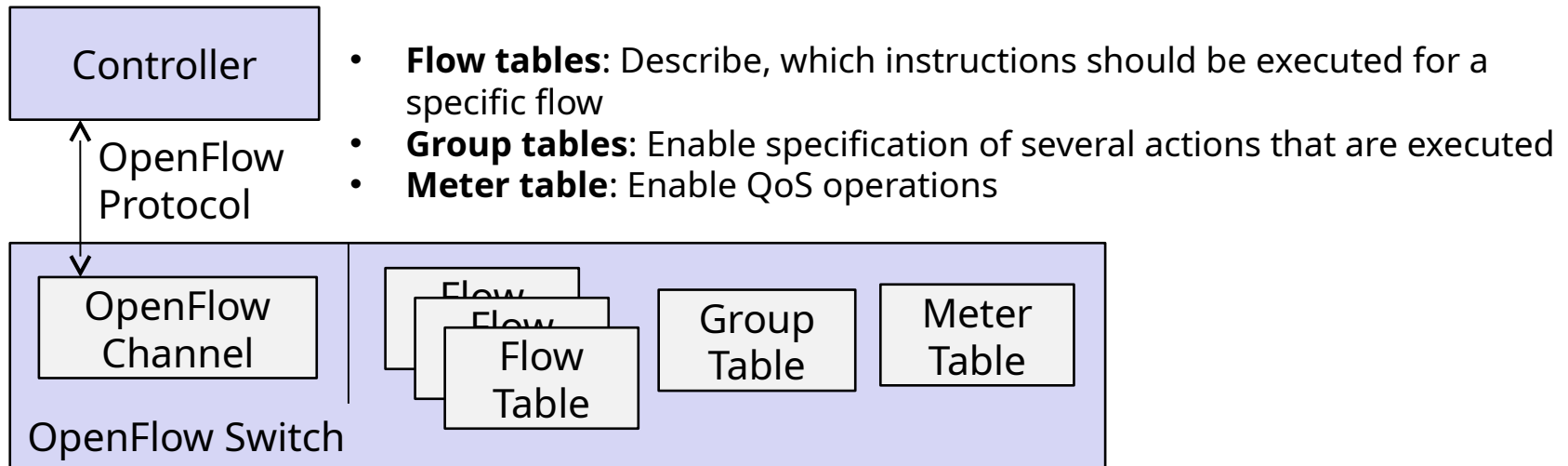
Southbound API

- API for controlling the data plane

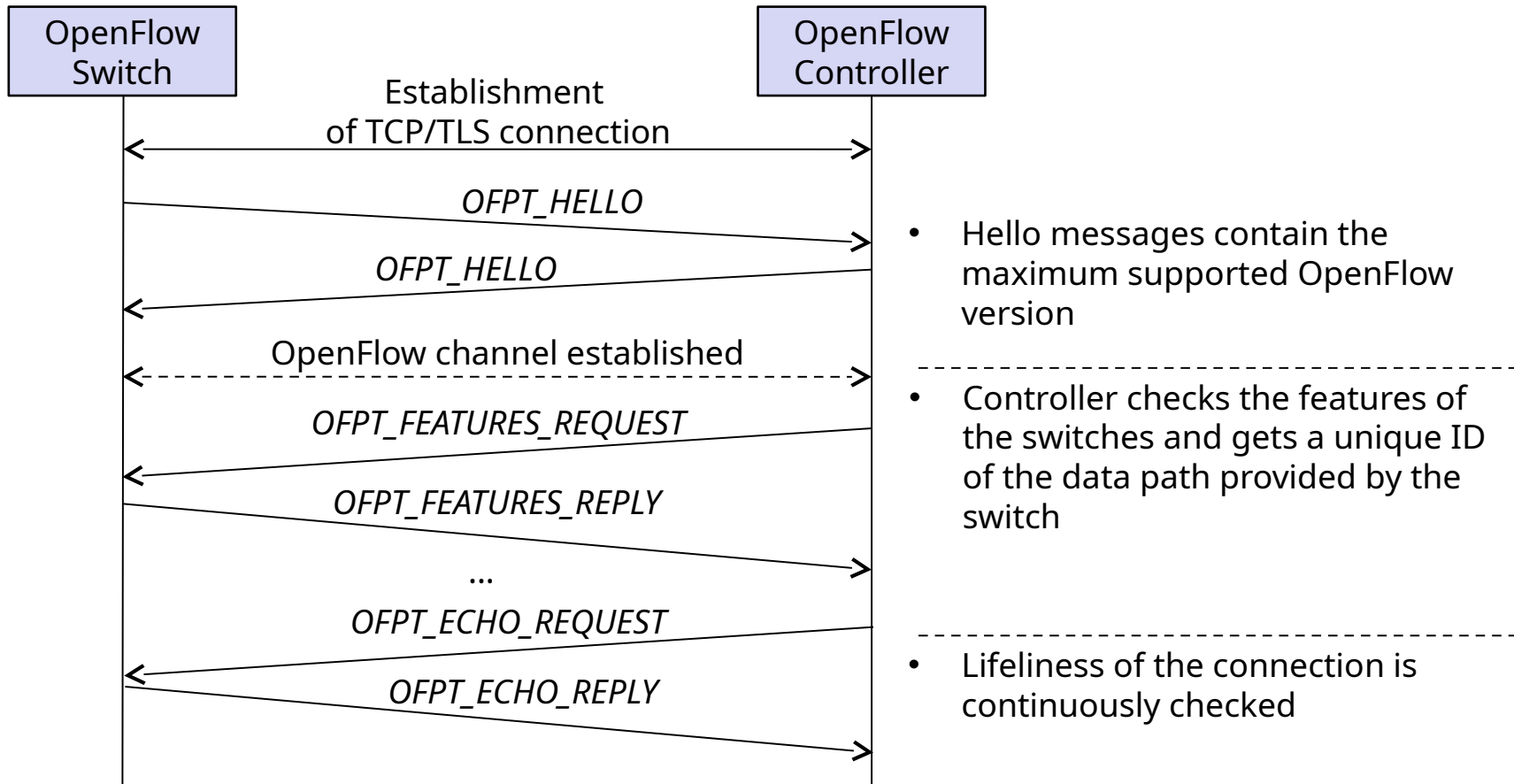
- Apart from being used by dedicated applications such as firewalls, the Northbound API may be used by virtualization solutions such as OpenStack Neutron
- Standard for Southbound API: **OpenFlow**

OpenFlow

- Controller manages entries in flow, group and meter table to determine behaviour of the data plane



OpenFlow – Channels



- OpenFlow channel is established before OpenFlow messages can be exchanged
- Each switch may establish channels to several controllers (e.g. for high-availability purposes)
- If there are several controllers, one of them may be configured as master

OpenFlow – Flow table

- Each table entry is structured as follows:

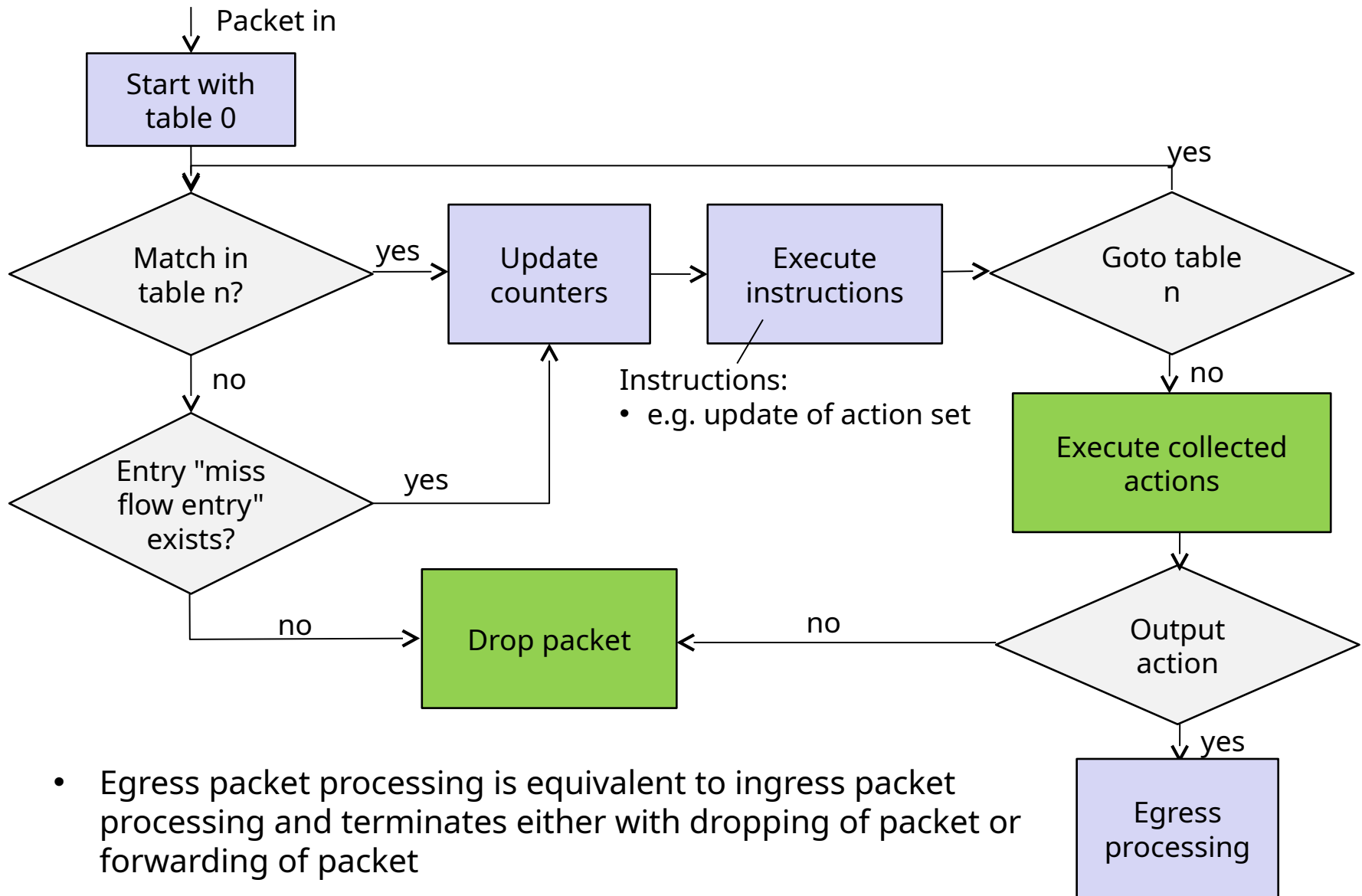
Match fields	Priority	Counters	Instructions	Timeouts	Cookie	Flags
--------------	----------	----------	--------------	----------	--------	-------

- Match fields: Information that is used to match packets
- Priority: Enables ranking of entries in case several entries match
- Counters: Incremented each time the entry is selected
- Timeouts: Maximum amount of time until the entry is removed by switch
- Cookie: Unique identifier which may be used by controller to refer to the entry
- Flags: Additional information which controls entry management

Possible **instructions** (= determine *steps of packet processing*):

```
enum ofp_instruction_type {  
  OFPIT_GOTO_TABLE = 1, /* Setup the next table in the lookup  
  pipeline */  
  OFPIT_WRITE_METADATA = 2, /* Setup the metadata field for use later in  
  pipeline */  
  OFPIT_WRITE_ACTIONS = 3, /* Write the action(s) onto the datapath action  
  set */  
  OFPIT_APPLY_ACTIONS = 4, /* Applies the action(s) immediately */  
  OFPIT_CLEAR_ACTIONS = 5, /* Clears all actions from the datapath  
  action set */  
  OFPIT_DEPRECATED = 6, /* Deprecated (was apply meter) */  
  OFPIT_STAT_TRIGGER = 7, /* Statistics triggers */  
  OFPIT_EXPERIMENTER = 0xFFFF /* Experimenter instruction */  
};
```

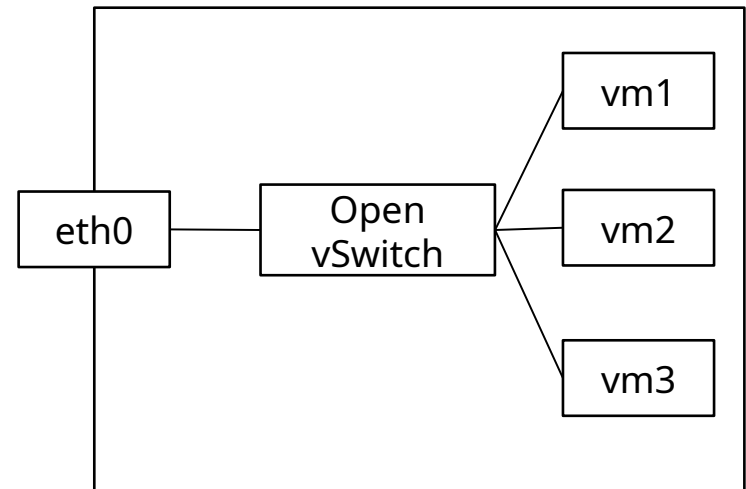
OpenFlow – Packet processing



- Egress packet processing is equivalent to ingress packet processing and terminates either with dropping of packet or forwarding of packet

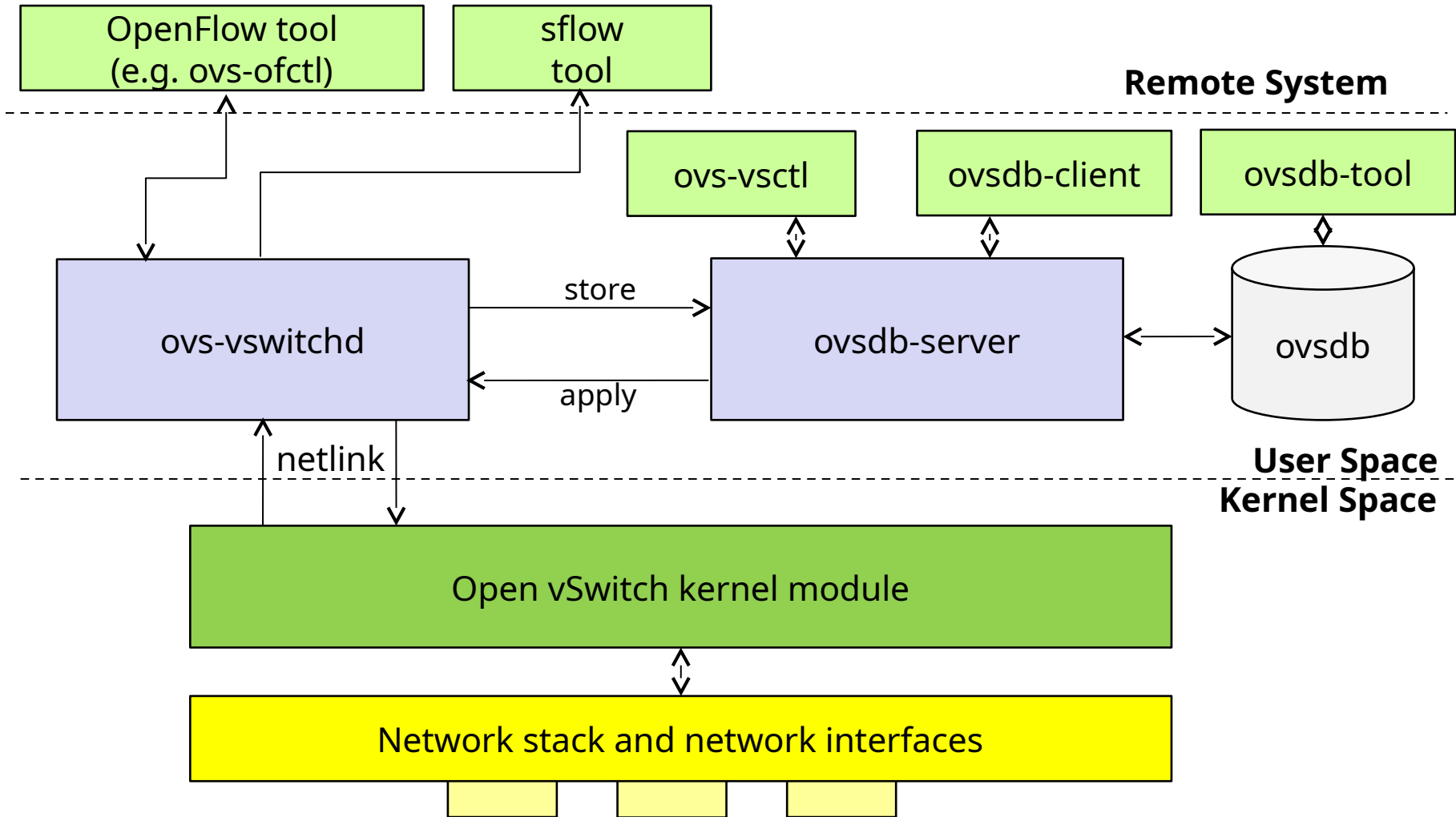
Open vSwitch

- Software implementation of a switch provided under Apache 2 license
- Focuses on Linux
- Supports several standards and functionalities, e.g.:
 - STP (IEEE 802.1D-1998)
 - Bonding / IEEE 802.3ad
 - sFlow / NetFlow / IPFIX
 - IPv6
 - VLANs / IEEE 802.1Q
 - VXLAN and further
 - OpenFlow
- Implementation provides a kernel module, daemons and command line tools for configuration and monitoring purposes



-
- Selected tools:
 - *ovs-vsctl*: Tool for managing the Open vSwitch daemon
 - *ovs-dpctl*: Tool for configuring the kernel data paths
 - *ovs-vlan-test*: Tool for VLAN issue analysis
 - *ovsdb-tool*: Tool for interacting with the Open vSwitch database
 - *ovs-ofctl*: Tool for interacting with the switch using the OpenFlow protocol

Open vSwitch - Architecture



- netlink: IPC mechanism for the communication between user space and kernel space, see RFC 3549
- For implementation details, see: <https://git.kernel.org/cgit/linux/kernel/git/stable/linux-stable.git/tree/net/openvswitch>

Open vSwitch

- Example: Output of **ovsdb-tool -m show-log**, after:
 1. A bridge has been created with `ovs-vsctl add-br br0`
 2. The bridge has been connected to veth0 via `ovs-vsctl add-port br0 veth0`

```
record 0: "Open_vSwitch" schema, version="8.3.0", cksum="3781850481 26690"

record 1: 2025-01-02 17:00:38.697 "ovs-vsctl (invoked by /bin/sh): ovs-vsctl --no-wait -- init -- set
Open_vSwitch . db-version=8.3.0"
  table Open_vSwitch insert row 3f797537:

record 2: 2025-01-02 17:00:38.709 "ovs-vsctl (invoked by /bin/sh): ovs-vsctl --no-wait set Open_vSwitch .
ovs-version=2.17.9 "external-ids:system-id=\"e03c588d-f6f4-4e44-8216-859393c3209d\""" "external-
ids:rundir=\"/var/run/openvswitch\""" "system-type=\"ubuntu\""" "system-version=\"22.04\"""
  table Open_vSwitch row 3f797537 (3f797537) diff:
...
record 5: 2025-01-02 17:00:38.826 "ovs-vsctl (invoked by /bin/sh): ovs-vsctl --no-wait add Open_vSwitch .
external-ids hostname=ubuntu-4gb-fsn1-1"
  table Open_vSwitch row 3f797537 (3f797537) diff:

record 6: 2025-01-02 17:01:16.184 "ovs-vsctl (invoked by -bash): ovs-vsctl add-br br0"
  table Interface insert row "br0" (d44114be):
  table Port insert row "br0" (c5ccce27):
  table Bridge insert row "br0" (1292383e):
  table Open_vSwitch row 3f797537 (3f797537) diff:
...
record 9: 2025-01-02 17:01:57.042 "ovs-vsctl (invoked by -bash): ovs-vsctl add-port br0 veth0"
  table Port insert row "veth0" (1e572e35):
  table Interface insert row "veth0" (80acbdf2):
  table Bridge row "br0" (1292383e) diff:
  table Open_vSwitch row 3f797537 (3f797537) diff:
...
```

Open vSwitch

- Communication with the switch can be done based on OpenFlow using ovs-ofctl, e.g. for querying information from the switch
- See for the below mentioned *OFPT_FEATURES_REPLY* message chapter 7.3.1 ("Handshake") of the Open vSwitch specification

#ovs-ofctl show br0

OFPT_FEATURES_REPLY (xid=0x2): dpid:00003e389212b642

n_tables:254, n_buffers:0

capabilities: FLOW_STATS TABLE_STATS PORT_STATS QUEUE_STATS ARP_MATCH_IP

*actions: output enqueue set_vlan_vid set_vlan_pcp strip_vlan mod_dl_src mod_dl_dst
mod_nw_src mod_nw_dst mod_nw_tos mod_tp_src mod_tp_dst*

1(veth0): addr:82:67:29:15:70:15

config: PORT_DOWN

state: LINK_DOWN

current: 10GB-FD COPPER

speed: 10000 Mbps now, 0 Mbps max

LOCAL(br0): addr:3e:38:92:12:b6:42

config: PORT_DOWN

state: LINK_DOWN

speed: 0 Mbps now, 0 Mbps max

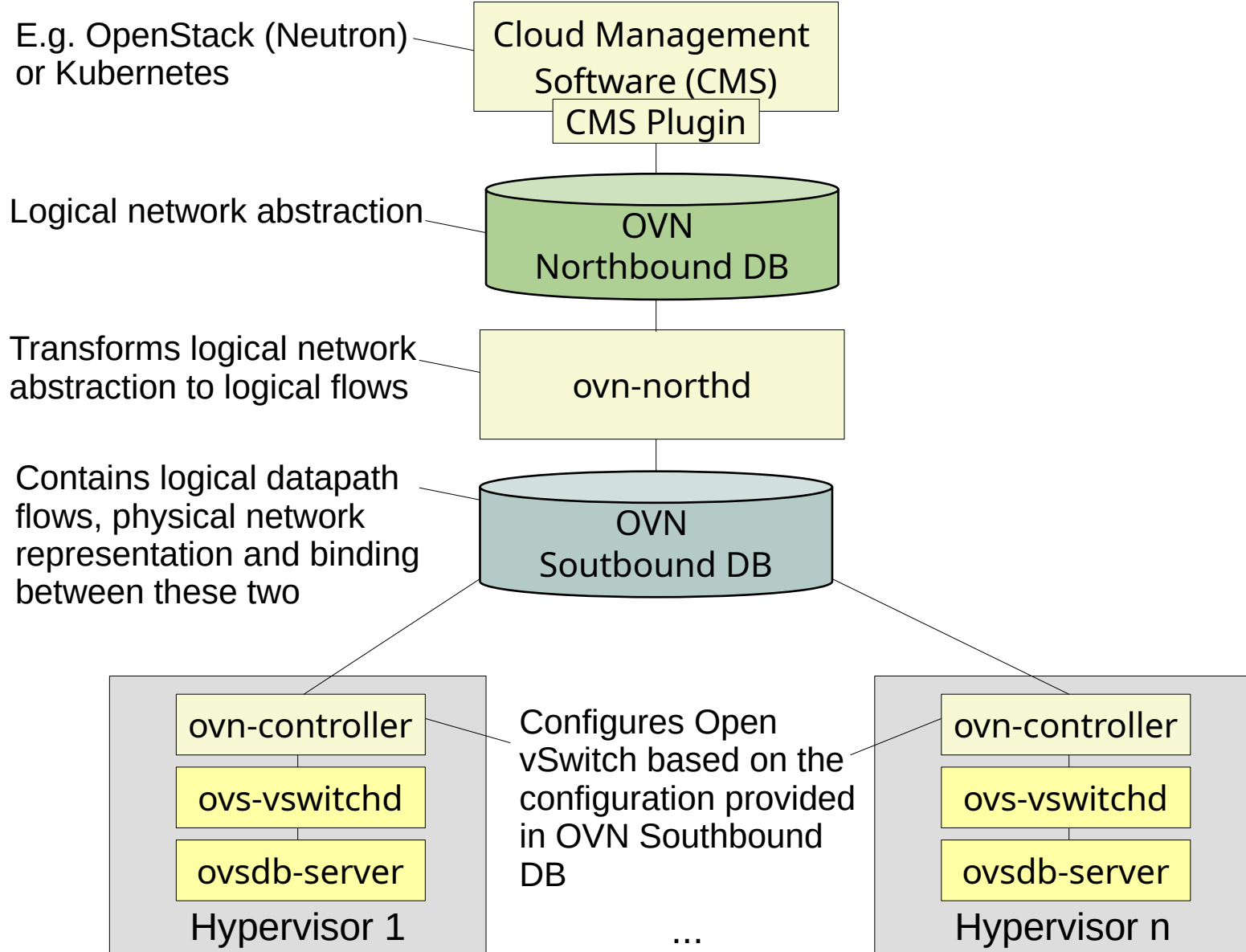
OFPT_GET_CONFIG_REPLY (xid=0x4): frags=normal miss_send_len=0

Open Virtual Network

- Open Virtual Network (OVN) introduces a high-level abstraction from Open vSwitch to configure routers and switches instead of flows
- Simple scenario with one switch and one router:

```
#ovn-nbctl ls-add s0
#ovn-nbctl lsp-add s0 s0p0
#ovn-nbctl lsp-set-addresses s0p0 "10:20:30:00:00:01 192.168.42.1"
#ovn-nbctl lr-add r0
#ovn-nbctl lrp-add r0 rp0 10:20:30:00:00:02 192.168.42.2/24
#ovn-nbctl lsp-add s0 rp0-attach
#ovn-nbctl lsp-set-type rp0-attach router
#ovn-nbctl lsp-set-addresses rp0-attach 10:20:30:00:00:02
#ovn-nbctl lsp-set-options rp0-attach router-port=rp0
#ovn-nbctl show
switch 99611a8b-43b3-4163-8615-26439b51b6c5 (s0)
  port s0p0
    addresses: ["10:20:30:00:00:01 192.168.42.1"]
  port rp0-attach
    type: router
    addresses: ["10:20:30:00:00:02"]
    router-port: rp0
router 28b73d61-eecf-4d9f-88a6-cdadea8af012 (r0)
  port rp0
    mac: "10:20:30:00:00:02"
    ipv6-lla: "fe80::1220:30ff:fe00:2"
    networks: ["192.168.42.2/24"]
```

Open Virtual Network - Architecture



References

- OpenFlow Switch specification (Version 1.5.1):
 - <https://www.opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.5.1.pdf>
- Open Networking Foundation:
 - <https://www.opennetworking.org>
- RFC of VXLAN specification:
 - <https://datatracker.ietf.org/doc/html/rfc7348>
- EVPN resources:
 - <https://datatracker.ietf.org/doc/html/rfc7209>
 - <https://datatracker.ietf.org/doc/html/rfc7432>
 - <https://datatracker.ietf.org/doc/html/rfc8365>
- Open vSwitch resources:
 - <http://openvswitch.org>
- OVN resources:
 - <https://www.ovn.org>